

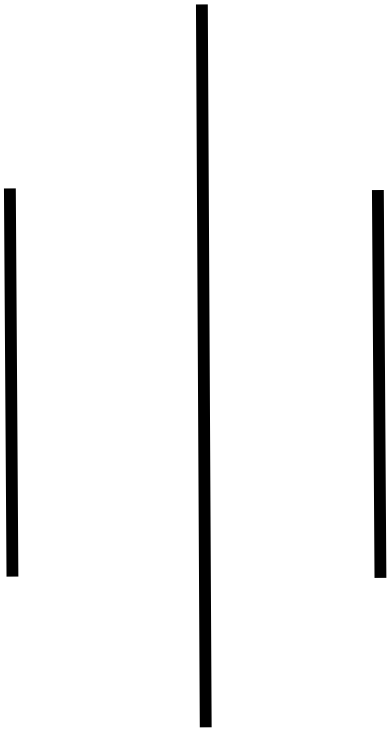

**Everest Engineering College**

**(Affiliated to Pokhara University)**

**Sanepa 02 Lalitpur**

**Department of Science and Technology**

**Course Name: Operating System design and implementation**



**Submitted By: Madhav Khanal**

**Submitted To: Kul Prasad Subedi**

# Assignment-1

## Question No.1

## Download the source code from this link: https://github.com/kpsubedi/OS/blob/master/linux/process_hw1.c. Answer the following

### A. comments the lines 8-10. compile and run the code. Submit the output of the program and explain why you receive the output. [10]

### Answer:

From given code after commenting the lines 8-10 I got the following output of the program.

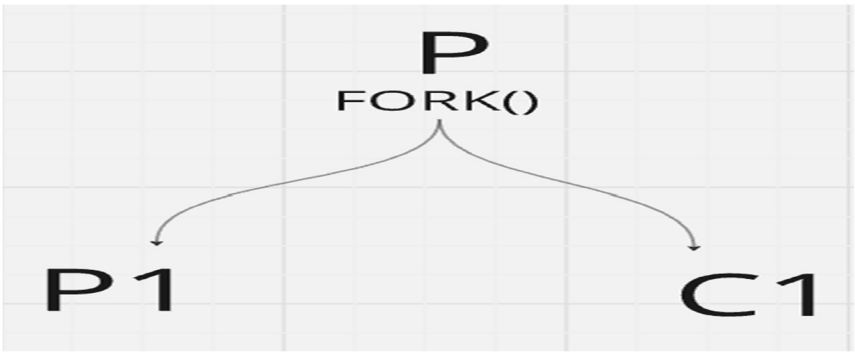
Output:

```
madhav@madhav-VirtualBox:~/Desktop/OS$ cd linux
madhav@madhav-VirtualBox:~/Desktop/OS/linux$ ls
assignments  isolation.txt  kernel.txt  process_hw1.c  process.txt
madhav@madhav-VirtualBox:~/Desktop/OS/linux$ gedit process_hw1.c
madhav@madhav-VirtualBox:~/Desktop/OS/linux$ gcc process_hw1.c -o question1
madhav@madhav-VirtualBox:~/Desktop/OS/linux$ ls
assignments  isolation.txt  kernel.txt  process_hw1.c  process.txt  question1
madhav@madhav-VirtualBox:~/Desktop/OS/linux$ question1
question1: command not found
madhav@madhav-VirtualBox:~/Desktop/OS/linux$ ./question1
I am the parent of pid=7048!
I am the child!
My parent process id = 7048!
madhav@madhav-VirtualBox:~/Desktop/OS/linux$
```

Explanation

**Process Tree:**

Each fork() doubles the number of processes:
**First fork():** Creates 2 processes.
P1: Original parent process.
C1: Child process created by P1.




**Behavior of fork():**

fork() creates a new child process.

The parent process receives the PID of the child process in the variable pid.

The child process receives 0 in the variable pid.

**Execution Flow:**

After fork(), two processes execute:

**Parent Process:**

Executes the block where pid > 0.

Prints: I am the parent of pid=7048! (where 7048 is the PID of the child process).

**Child Process:**

Executes the block where pid == 0.

Prints: I am the child! and My parent process id = 7048! (where 7048 is the PID of the parent process).

**Concurrency:**

The two processes execute concurrently, so the order of their outputs may vary.

Typically, the parent process finishes first, but this is not guaranteed.

**Key Points to Note**

Only one child process is created because a single fork() is called because comment insert to line no 8-10

The parent process prints information about the child, while the child process prints its own details.

The total number of processes running is 2 (parent and child).

## B. comment the lines 9-10. compile and run the code. Submit the output of the program and explain why you receive the output. [10]

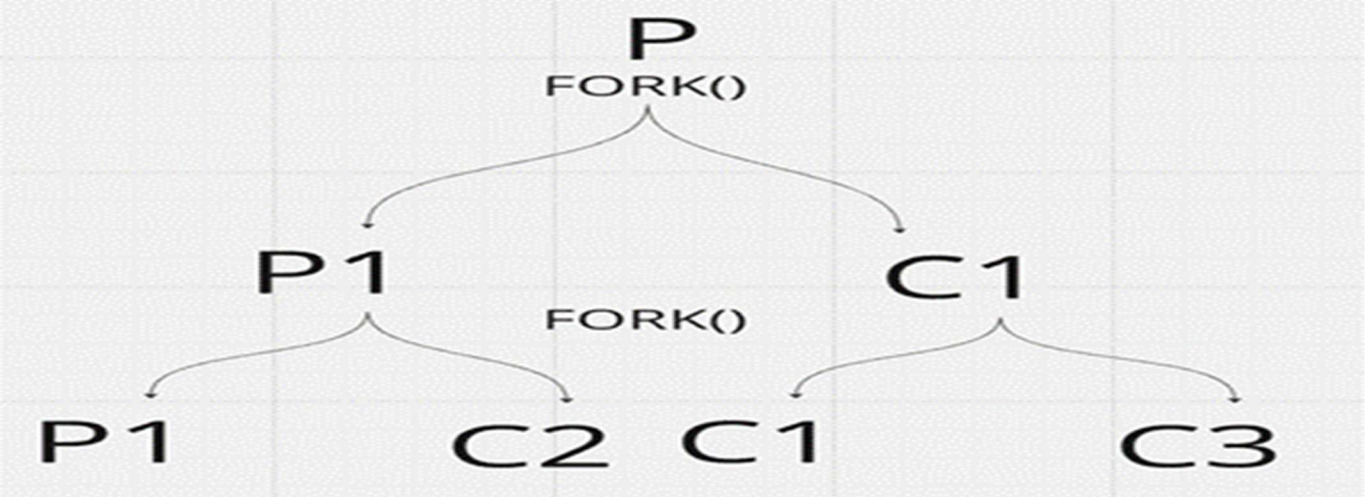
## Answer:

From given code after commenting the lines 9-10 I got the following output of the program.

```
madhav@madhav-VirtualBox:~/Desktop/OS/linux$ gedit process_hw1.c
madhav@madhav-VirtualBox:~/Desktop/OS/linux$ gcc process_hw1.c -o question2
madhav@madhav-VirtualBox:~/Desktop/OS/linux$ ls
assignments  isolation.txt  kernel.txt  process_hw1.c  process.txt  question1  question2
madhav@madhav-VirtualBox:~/Desktop/OS/linux$ ./qusetion2
bash: ./qusetion2: No such file or directory
madhav@madhav-VirtualBox:~/Desktop/OS/linux$ ./question2
I am the parent of pid=7524!
I am the child!
My parent process id = 7524!
madhav@madhav-VirtualBox:~/Desktop/OS/linux$ I am the parent of pid=7525!
I am the child!
My parent process id = 7525!
```

**Process Tree**

Each fork() doubles the number of processes:
**First fork():** Creates 2 processes.
P1: Original parent process.
C1: Child process created by P1.
**Second fork():** Both P1 and C1 execute this fork() independently, creating:
P1 creates C2.
C1 creates C3.
**Process Flow Tree:**




**Execution Flow**

P1 (Parent Process):

Executes both fork() calls.

After the first fork, it becomes the parent of C1.

After the second fork, it becomes the parent of C2.

C1 (Child of P1 from First Fork):

Executes the second fork() and becomes the parent of C3.

C2 (Child of P1 from Second Fork):

Does not execute any further fork() calls (no new children created).

C3 (Child of C1 from Second Fork):

Does not execute any further fork() calls (no new children created).

## C. comment the lines 10. compile and run the code. Submit the output of the program and explain why you receive the output. [10]

## Answer:

From given code after commenting the lines 10 I got the following output of the program.

```
madhav@madhav-VirtualBox:~/Desktop/OS/linux$ gedit process_hw1.c
madhav@madhav-VirtualBox:~/Desktop/OS/linux$ gcc process_hw1.c -o question3
madhav@madhav-VirtualBox:~/Desktop/OS/linux$ ls
assignments  isolation.txt  kernel.txt  process_hw1.c  process.txt  question1  question2  question3
madhav@madhav-VirtualBox:~/Desktop/OS/linux$ ./question3
I am the parent of pid=8171!
I am the child!
My parent process id = 8172!
I am the parent of pid=8174!
I am the child!
I am the parent of pid=8172!
My parent process id = 8174!
I am the parent of pid=8175!
I am the child!
I am the child!
My parent process id = 8171!
My parent process id = 8175!
madhav@madhav-VirtualBox:~/Desktop/OS/linux$
```

Process Tree Analysis

First fork(): Creates 2 processes:

P1: Original parent process.

C1: Child process created by P1.

Second fork(): Both processes (P1 and C1) execute this fork():

P1 creates C2.

C1 creates C3.

At this point, there are 4 processes: P1, C1, C2, C3.

Third fork(): All existing processes (P1, C1, C2, C3) execute this fork(). Each creates one child:

P1 creates C4.

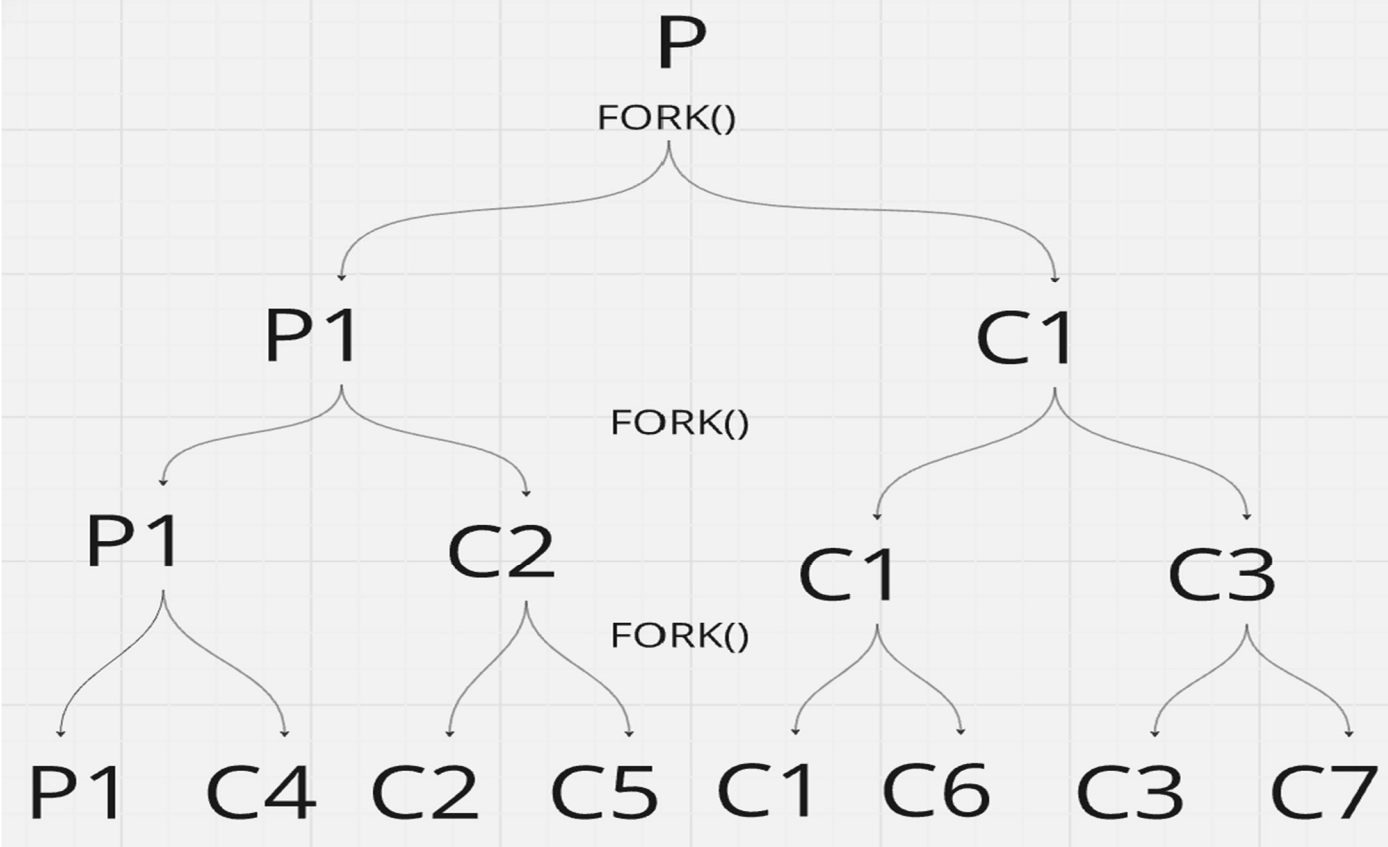
C1 creates C5.

C2 creates C6.

C3 creates C7.

At this point, there are 8 processes: P1, C1, C2, C3, C4, C5, C6, C7.

**Process Flow Tree:**




## Execution Flow

The program starts with only 1 process:
P (Parent Process, PID of the program).
**First fork()**

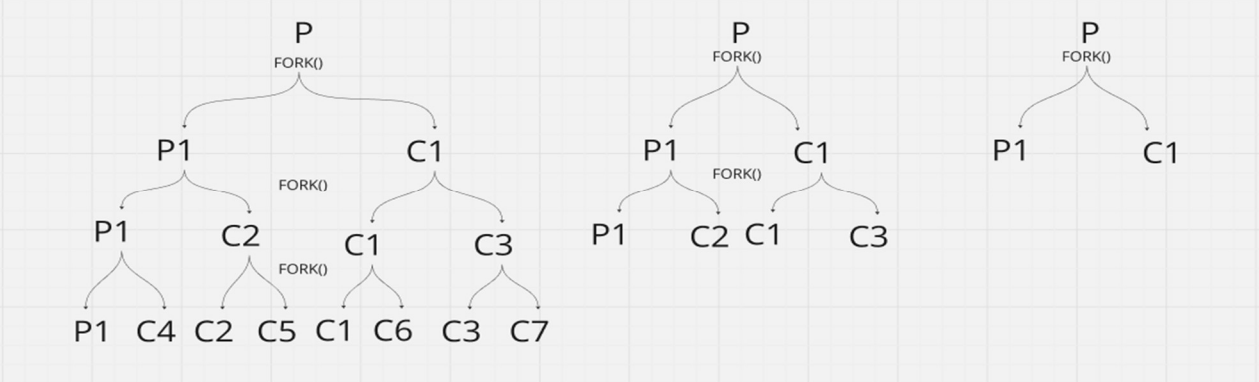
The first fork() creates a new child process, C1. Now, there are 2 processes:
P1: The original process.
C1: A new child process created by P.
Both P1 and C1 continue executing the same code from the point after the first fork().
Each process independently executes the subsequent code.
**Second fork()**

Both P1 and C1 execute the second fork() independently, creating 2 new child processes:
P1 creates C2.
C1 creates C3.
Now, there are 4 processes:
P1 (Parent process).
C1 (Child of P1 from the first fork).
C2 (Child of P1 from the second fork).
C3 (Child of C1 from the second fork).
All 4 processes (P1, C1, C2, and C3) continue executing the subsequent code from the point after the second fork().

**Third fork()**

All 4 processes (P1, C1, C2, and C3) execute the Third fork(), each creating a new child process:
P1 creates C4.
C1 creates C5.
C2 creates C6.
C3 creates C7.
Now, there are 8 processes in total:
P1, C1, C2, C3 (existing processes).
C4, C5, C6, C7 (new child processes created in the Third fork()).
All 8 processes independently execute the remaining code, including the if-else block.

**Process Flow Tree Overall Code:**




## Q2. Execute the command :() {:|:&};: on the Linux VM. Explain how this command works. [20]

: IS function name

Ls & run program in background and recursive and use for maximum resource

Os three types

Windows @ p file

Linux  @ELF(binary file)

Iso @macro

magicbyte

definitation of ls in os

**which ls**

**strace ls**

**fork() to create process of making child**

**execve load child process code to parent**

**l called pipe two program link and take input to next of frist output**

**like**

**ls l grep madhav**

**take input**

ps -ef l program name vim and see process id

kill –l

to find bug

is there bug or not go to website (canyouhack.us)

study how web browser work

go to developer tools

view page

**sildes and books for operating system**

https://pages.cs.wisc.edu/~remzi/OSTEP/

https://wiki.osdev.org/Expanded_Main_Page

https://www.os-book.com/

**https://www.os-book.com/OS10/slide-dir/index.html@ for slides**

https://github.com/kpsubedi/computer_architecture/blob/master/linux_assembly/lab_setup

https://github.com/kpsubedi/computer_architecture/blob/master/linux_assembly/assembly-32/what_is_assembly.org

https://github.com/kpsubedi/computer_architecture/blob/master/prerequisite/gdb_guide

https://github.com/kpsubedi/computer_architecture/blob/master/prerequisite/tools

**Important links and materials for threads**

**@ course study for security**

**https://cs155.stanford.edu/**

**@ to find bug go to**

**https://canyouhack.us/**

**@ notes on threads**

**https://learn.microsoft.com/en-us/windows/win32/procthread/fibers**

**@ to check dll file go to**

**https://www.virustotal.com/gui/home/upload**

**Internal Exam(Class exam)**

**1. What is an Operating System? What are the implementation types of Kernels and discuss each of the implementation types?**

An **Operating System (OS)** is system software that acts as an intermediary between computer hardware and the user, facilitating tasks like memory management, process management, file management, and hardware interaction. It ensures efficient resource allocation and provides a user interface.

**Kernel Types:**

1. **Monolithic Kernel:**
    - All components (e.g., memory management, process scheduling, device drivers) run in kernel space.
    - **Pros:** Efficient due to direct interaction between components.
    - **Cons:** Large and complex, making maintenance harder.
    - **Example:** Linux, Unix.
2. **Microkernel:**
    - Minimizes kernel functions, moving most services to user space (e.g., device drivers).
    - **Pros:** More modular, easier to maintain, and more secure.
    - **Cons:** Slightly slower due to context switching between kernel and user space.
    - **Example:** Minix, QNX.
3. **Hybrid Kernel:**
    - Combines features of both monolithic and microkernels for balance in performance and modularity.
    - **Pros:** Better performance than microkernels while maintaining modularity.
    - **Cons:** Complexity of design and implementation.
    - **Example:** Windows NT, macOS (XNU).
4. **Exokernel:**
    - Provides minimal abstractions and leaves resource management to applications.
    - **Pros:** Maximizes performance and flexibility.
    - **Cons:** More complexity for application developers.

- **Example:** Aegis (academic).

---

**2. Compare and contrast between process and thread. Can we achieve isolation using process and thread?**

- **Process:**
  - Independent program with its own memory space and resources.
  - **Isolation:** Strong isolation as processes don't share memory.
  - **Pros:** Security and stability since one process cannot directly interfere with another.
  - **Cons:** Heavier, with high context switching overhead.
- **Thread:**
  - A smaller unit of execution within a process that shares the same memory space.
  - **Isolation:** Threads share resources, providing less isolation.
  - **Pros:** Lighter and faster than processes, suitable for tasks requiring shared data.
  - **Cons:** Less isolation, making synchronization necessary to avoid conflicts.

**Isolation Comparison:**

- **Processes** offer **stronger isolation** because they have separate memory spaces, whereas **threads** lack full isolation due to shared memory. Threads require synchronization to ensure they don't interfere with each other's tasks.

---

**3. Discuss different types of hypervisor architecture with examples.**

**Hypervisor** enables virtualization, allowing multiple operating systems to run on the same physical hardware.

1. **Type 1 Hypervisor (Bare-metal):**
   - Runs directly on hardware without an underlying OS.
   - **Pros:** Better performance and security, as it directly manages hardware.
   - **Cons:** Less flexible than Type 2.
   - **Example:** VMware ESXi, Microsoft Hyper-V.
2. **Type 2 Hypervisor (Hosted):**
   - Runs on top of a host OS, using it for hardware resource management.

- **Pros:** Easier to set up and use, especially on personal machines.
- **Cons:** Slower performance due to dependency on the host OS.
- **Example:** VMware Workstation, Oracle VirtualBox.

3. **Hybrid Hypervisor:**
   - Combines features of both Type 1 and Type 2 hypervisors, running on a host OS but offering near-bare-metal performance.
   - **Pros:** Offers flexibility with performance optimizations.
   - **Cons:** More complex than both Type 1 and Type 2.
   - **Example:** Microsoft Hyper-V.

---

**4. Why do we need Mandatory Access Control (MAC) in addition to Discretionary Access Control (DAC)? Explain the components of the SELinux MAC system.**

**Mandatory Access Control (MAC)** is essential for enhanced security beyond the user-defined permissions offered by **Discretionary Access Control (DAC)**. While DAC allows users to control access to their resources, it relies on user discretion and can lead to misconfigurations or malicious actions. **MAC** enforces security policies at a system level, making it impossible for users to override these controls.

**SELinux Components:**

1. **Security Labels:**
   - Objects (files, processes) are assigned labels that define their security context (e.g., user, role, type, and level).
2. **Security Context:**
   - A combination of security labels attached to subjects (processes) or objects (files). It defines how they can interact under the SELinux policy.
3. **Policy Rules:**
   - SELinux uses predefined policies to control interactions between subjects and objects based on their security labels. These policies are used to determine what actions (e.g., read, write) are permitted.
4. **Access Control:**
   - SELinux compares the security context of processes and objects, granting or denying access based on the security policy.
5. **Enforcement Modes:**

- **Enforcing:** SELinux policies are actively enforced.
- **Permissive:** SELinux logs violations but does not block access.
- **Disabled:** SELinux is turned off.

**Why MAC is necessary:** While DAC grants user flexibility, it can be vulnerable to misuse. MAC, on the other hand, ensures more rigid control, reducing the risk of unauthorized access and enhancing system security.